Rendering and Optimization

The frame budget, the three trees, and finding the bottleneck before you fix it

Dinu-Ștefan Rusu
FILS, Universitatea Politehnica București · Spring 2027

TODAY

What you should leave with

Spend the frame budget

One frame is 1 ÷ refresh rate. See where Impeller and the raster thread spend it.

Read the three trees

Widget, Element and RenderObject, and how reconciliation decides what a rebuild actually touches.

Cut real costs

const constructors, builder lists, and when an offscreen buffer or a decoded image is the actual problem.

Profile before you fix

Read the DevTools frame chart, tell a UI-thread problem from a raster-thread one, and fix a real list screen.

PART 1 · PERFORMANCE

Two things we call performance

The frame budget, and where the milliseconds go

PERFORMANCE

Speed and efficiency are different problems

Speed: how fast the app answers. Startup latency, scroll smoothness, the delay between a tap and something moving. The user feels this directly, in milliseconds.

Efficiency: what the app costs to run. Memory footprint, install size, and energy. The user feels this indirectly, as a hot phone and a flat battery.

This lecture

Rendering and optimization are mostly about speed. Lecture 7 and Lecture 14 cover the energy half of efficiency in depth.

A slow app and an expensive app fail differently. Keep the two apart when you read a bug report: ”laggy” is speed, ”drains the battery” is efficiency.

PERFORMANCE

Startup latency: cold, warm, hot

START	WHAT THE SYSTEM ALREADY HAS	VITALS ABOVE	FLAGS
Cold	nothing: process created, runtime initialized, first frame drawn	5 s	
Warm	the process, but the screen is rebuilt from scratch	2 s	
Hot	process and UI both alive, just brought to the foreground	1.5 s	

Measure the one your users actually hit. Cold start is what a store reviewer sees once. Hot start is what a daily user sees ten times a day.

PERFORMANCE

One frame budget = 1 ÷ refresh rate

16.7 ms

60 Hz: the old assumption, still true on older devices

11.1 ms

90 Hz: common mid-range phones in 2026

8.3 ms

120 Hz: most 2026 flagships

A frame budget is not a constant. It is one divided by the refresh rate, and that rate has climbed for a decade.
Inside it, the UI thread runs build, layout and paint, then hands off to the raster thread and the GPU.
Jank is variance, not average. A steady 30 fps feels smoother than 55 fps with occasional spikes.

Budget for the fastest panel you support, not the slowest. Lecture 2 introduced this budget for concurrency; this lecture spends it on rendering.

PART 2 · RENDERING

Flutter owns every pixel

Three trees, reconciliation, and what each operation costs

Flutter owns every pixel; Impeller draws it

Flutter does not draw the platform's native widgets. It ships a renderer and paints the entire surface itself, so every rendering problem in it is Flutter's, and yours.

The renderer is **Impeller**, not Skia. It became the default on iOS in 2023 and on Android soon after, and Skia is now fully removed from both, except a fallback for very old Android devices.

Why it exists

Skia compiled a shader the first time an effect appeared: a stutter on first use. Impeller compiles every shader at build time instead.

The pipeline, in order. Your widgets → the framework (layout, paint, layers) → Impeller (precompiled shaders) → Metal or Vulkan.

Shader warm-up: dead code under Impeller

Under Skia, a shader was compiled the first time the GPU needed it, mid-frame. The classic symptom was ”first-run jank”: an effect that stuttered once, the first time it appeared, then never again.

The workaround was shader warm-up: bundling a set of SkSL shaders and compiling them during a splash screen, before the user could trigger the expensive first frame.

Impeller compiles every shader ahead of time, at build time. The first frame that uses an effect costs the same as the thousandth, so there is no first-run jank left to warm up.

If a tutorial tells you to warm up shaders, it predates Impeller. Treat any Flutter rendering advice written before 2023 as needing a second check against current facts.

Three trees, three very different costs

ADMD Lectures 3 and 5 introduced the three trees at the developer level. Here is what each one actually costs to touch.

Widget tree

The blueprint. Immutable, no behavior. Allocating one is a bump-pointer write: constant time, genuinely cheap.

Element tree

The mediator. Persists across frames, holds your State, decides if the old render object can be kept.

RenderObject tree

The machinery. Layout, paint, hit-testing. Expensive to touch, kept alive as long as possible.

Flutter does not rebuild the whole widget tree every frame. Only dirty subtrees rebuild. ”Cheap to rebuild” is about allocation, not permission to rebuild a screen for one changed label.

StatelessWidget does not build once

A common misconception: a `StatelessWidget` builds once and stays that way. It does not. It rebuilds whenever its parent rebuilds, or whenever an `InheritedWidget` it depends on notifies a change. ”Stateless” describes what it holds, not how often `build()` runs.

	STATELESS	STATEFUL
build() runs	on insert, on every parent rebuild	all of that, plus every setState()
holds	no mutable data of its own	a State object, kept across rebuilds

Stateless is not static. It just does not own the state that changes. Something above it still decides when it runs.

Reconciliation: runtimeType first, then key

```
// The whole reuse decision:
bool canUpdate(Widget a, Widget b) {
  return a.runtimeType == b.runtimeType
      && a.key == b.key;
}
// true  -> Element updated in place,
//          render object REUSED.
// false -> old subtree disposed.
```

The element tree walks the new children in order and asks this once per child: linear, $O(N)$ in the child count.

Keeping types and keys stable is the whole game: it keeps the answer ”true”.

Same type, same key: the expensive object survives. Swapping a `Container` for a `SizedBox` in a conditional is a type change, and it silently destroys the state below it.

What setState actually does

`setState` marks exactly one Element dirty and asks for a frame. On that frame, `build()` runs on the dirty subtree, and on nothing else.

Sibling subtrees are never visited. Their Elements, State objects and render objects are left untouched.

The cost of a `setState` is the size of the subtree beneath it. Push it as far down the tree as you can.

1. `setState(() { ... })`: mutate a field
2. `markNeedsBuild()`: this Element is dirty
3. next frame: `build()` on dirty subtrees only
4. reconcile children: runtimeType, then key
5. layout and paint only what changed

Only dirty subtrees rebuild. That is the whole performance model this lecture builds on.

Keys: identity for things that move

Without a key, children are matched by position. Reorder a list and Flutter hands item three's state to whatever now sits third.

With a key, Flutter recognizes the move: it relocates the element and its render object instead of rebuilding them.

```
for (final t in todos)
  TodoTile(key: ValueKey(t.id), todo: t)
```

Which key

`ValueKey(item.id)` for data you own. `ObjectKey` for identity by instance. `UniqueKey` only for destruction: it never matches.

Delete the first row without a key, and the second row inherits its state. Its scroll offset and its animation jump to the wrong item.

GlobalKey: identity across the whole tree

A `GlobalKey` can reparent a widget: move a live element under a completely different parent, keeping its state.

Three costs come with that power, every time it fires.

1. detach and re-attach, with a lookup in a global registry
2. layout and paint invalidated in both the old and the new location
3. every InheritedWidget dependency below it resolved again

Keys are cheap. GlobalKeys are not. Use a GlobalKey when you actually need to reparent or reach a State from outside it, not as a convenient handle to a widget.

What each operation actually costs

OPERATION	COST	WHY
Building a widget	constant	an allocation in the young heap, nearly free to collect
Reconciling children	linear, $O(N)$	one canUpdate call per child
GlobalKey reparenting	search + re-attach	invalidates layout in two places
Intrinsic sizing (nested)	quadratic, $O(N^2)$	each level walks its subtree again

Layout is one depth-first pass: constraints go down, sizes come up. A relayout boundary stops a dirty flag from propagating upward. A fixed-size box around a busy widget is a real optimization.

PART 3 · OPTIMIZATION

Small changes, large effects

const, builder lists, offscreen buffers, and images

const constructors, and what they save

```
// Rebuilt, re-diffed on every build:
Padding(
  padding: EdgeInsets.all(16),
  child: Text('Total'),
)
// The SAME instance, forever:
const Padding(
  padding: EdgeInsets.all(16),
  child: Text('Total'),
)
```

Dart canonicalizes const expressions: identical const widgets are the same object.

Reconciliation then returns immediately, and the subtree below is skipped.

It needs every argument to be compile-time constant, a reason to hoist styles into static fields.

This is the one optimization that costs nothing but a keyword. Turn on the `prefer_const_constructors` lint and let the IDE add them for you.

Lists: build what is on screen, nothing else

The eager alternative, `ListView(children: ...)`, builds every child up front.

```
// Lazy: builds only what is visible.
ListView.builder(
  itemCount: items.length,
  itemExtent: 72,      // O(1) scroll math
  itemBuilder: (context, i) => RowTile(
    key: ValueKey(items[i].id),
    item: items[i],
  ),
)
```

Startup: frame one skips builds it will never show.

Memory: elements track the viewport, not the data.

Scrolling: `itemExtent` makes it arithmetic.

The bad version is not slow, it is unbounded. Works on ten test rows, fails on a user's ten thousand.

Offscreen buffers: when a layer helps and hurts

Opacity and ClipRRect can force a saveLayer:

1. allocate an offscreen buffer in GPU memory
2. render the entire child subtree into it
3. apply the alpha or the clip to the finished texture
4. composite the texture back onto the frame

Four GPU steps, every frame, for one faded widget: the classic raster-thread spike.

Animating a fade: use `AnimatedOpacity`, which drives a compositor layer instead of re-diffing the subtree each frame.

Hiding something: `Opacity(opacity: 0)` still lays out and paints. Use `Visibility`, or do not build the widget.

Rounded corners: a `BoxDecoration` paints the shape directly; reach for `ClipRRect` only to clip real content.

RepaintBoundary: the mirror image

`RepaintBoundary` deliberately creates a layer, so a repaint inside it does not spread to the widgets around it.

The cost is video memory: width times height times 4 bytes, for RGBA, per boundary.

Wrapping every row of a list fragments the layer tree and multiplies that cost by the row count.

⚠ Wrap the right thing

Wrap the one spinner that animates, not the list around it. A boundary around static content buys nothing.

A layer is a deliberate trade: less repaint spread, for more video memory. Reach for it once you have profiled a real repaint problem, not by default.

OPTIMIZATION

Images and memory: the decoding cost

Decoding an image allocates a full bitmap at its native pixel dimensions, not at the size it is displayed on screen.

A large photo decoded to fill a small avatar still allocates the full decoded bitmap first, then Flutter scales it down for display.

A screen with many such images, a photo grid or a long avatar list, can allocate far more memory than the screen actually shows, and force needless garbage collection.

The source resolution decides the memory cost, not the layout size. A 4000-pixel-wide photo in a 40-pixel avatar still pays for 4000 pixels of decoded bitmap, unless you tell Flutter otherwise.

OPTIMIZATION

cacheWidth and precacheImage

```
// Decodes at the target size:
Image.network(
  user.avatarUrl, cacheWidth: 96,
)
// Decodes ahead of time:
precacheImage(
  NetworkImage(nextUrl), context,
);
```

`cacheWidth` cuts decoded memory roughly by the square of the resize ratio.

Flutter also keeps a small in-memory cache, so a re-shown image does not decode twice.

Decode once, at the size you show. `precacheImage` moves the decode off the frame that would otherwise pay for it, such as after a navigation.

PART 4 · PROFILING

Find the bottleneck first

The DevTools performance view, and a before-and-after example

PROFILING

Profile a real build, not a debug one

```
# Debug builds are JIT and assert-heavy.
# Never quote a debug-mode number.
flutter run --profile        # on a real device
# then open DevTools -> Performance
```

Profile mode compiles close to release performance but keeps the tracing information DevTools needs.

Run it on a real device. A simulator has different GPU and thermal behavior, so its numbers do not transfer.

Profile before you change anything. A guess about what is slow is usually wrong, and always unverifiable without a trace.

PROFILING

The frame chart: one bar per frame

The Performance view's timeline draws one bar per frame, split into a UI-thread segment and a raster-thread segment.

A bar taller than the budget line, 8.3 ms at 120 Hz, marks a janky frame. The timeline scrolls as the app runs, so a scroll gesture that jumps the line is easy to spot.

Click a bar to expand it into a flame chart of the actual calls that ran during that frame: build, layout, paint, and everything beneath them.

The shape of the bar is the diagnosis. A tall, wide band across many frames is a systemic problem. One isolated tall bar is usually a single expensive event, like a navigation.

PROFILING

Blue bar or green bar: which thread, which fix

SIGNAL	WHAT IT MEANS
Tall blue bar (UI thread)	your Dart is slow: an expensive build(), or isolate-bound work
Tall green bar (raster thread)	the GPU is slow: a saveLayer, an oversized image, too many layers
One widget lights up every frame	seen with ”Track widget builds”; a whole screen for one label is the bug
Deep stack under performLayout	layout thrashing, often from nested IntrinsicWidth or Height

The color of the bar tells you which half of the problem you have. Fix blue with less work in build(); fix green with fewer or smaller layers.

Before: a list screen that drops frames

```
ListView(children: users.map((u) =>
  Opacity(
    opacity: 1,
    child: ListTile(
      leading:
    Image.network(u.avatarUrl),
      title: Text(u.name),
    ),
  ),
).toList())
```

1,200 users. Every row builds up front, before frame one shows.

Avatars decode at full source size, not tile size.

`Opacity(opacity: 1)` still forces a saveLayer per row.

The frame chart: a steady band of tall blue bars, plus green spikes on scroll. Three problems, stacked in one screen.

After: the fix, and what actually moved

```
ListView.builder(
  itemCount: users.length,
  itemExtent: 72,
  itemBuilder: (context, i) => ListTile(
    key: ValueKey(users[i].id),
    title: Text(users[i].name),
  ),
)
```

Only visible rows build. `ValueKey` keeps state with the right user.

The avatar now loads with `cacheWidth: 96`, and the no-op `Opacity` is gone.

Each fix maps to an earlier slide: builder lists, keys, decoding. The blue band drops under budget, and the green spikes disappear.

RECAP

Three things to remember

1. **One frame budget is 1 divided by refresh rate.** 8.3 ms at 120 Hz, not a fixed 16 ms; the budget itself moves with the display.
2. **Reconciliation, not full rebuilds.** Same runtimeType and same key keep the render object. const, keys and ListView.builder are how you keep that answer true.
3. **Profile before you fix.** The frame chart's color tells you whether the problem is your Dart, blue, or the GPU, green.

FURTHER READING

docs.flutter.dev/perf/best-practices · DevTools: Performance view · Rendering performance · Impeller

Questions?

dinu_stefan.rusu@upb.ro · Microsoft Teams: course channel